InferTutor Arena

Capstone Project for the Inference Engineering Workshop

Vizuara AI Labs - Inference Engineering Workshop

InferTutor Arena

Capstone Project for the Inference Engineering Workshop

You have spent the course studying the pieces of modern inference systems: KV cache behavior, MQA/GQA/MLA, FlashAttention, PagedAttention, prefix caching, chunked prefill, quantization, speculative decoding, batching, parallelism, vLLM, observability, and production serving.

Now you will compose those ideas into one working system.

Your mission is to launch InferTutor, a personalized AI tutor for inference engineering students. Learners will ask conceptual questions, upload simple diagrams, ask for debugging help, and request optimization advice. Your system must answer correctly while serving many users with low latency and high throughput.

This is not a notebook-only assignment. You will deploy a real OpenAI-compatible vLLM server on Modal, run load tests, measure TTFT, ITL, throughput, errors, and GPU efficiency, then improve the system through careful inference engineering.

The Product Story

Imagine this is the final week of the workshop. Hundreds of students are revising at the same time. They ask questions like:

- "Why does p95 TTFT spike when I increase concurrent users?"
- "Should I use tensor parallelism or replicas for this 4B tutor model?"
- "Look at this toy serving diagram. Is this workload prefill-heavy or decode-heavy?"
- "How do prefix caching and chunked prefill interact?"

The user experience must feel responsive. The first token should arrive quickly. The stream should not stutter. The system should not collapse under a burst of users. The answer quality should remain useful.

You are the inference engineer responsible for this launch.

Model Choice

We will use:

```
Qwen/Qwen3-VL-4B-Instruct
```

Why this model:

1. It is multimodal, so the capstone can include both text and image requests. 2. It is small enough to run on a single H100 while still being realistic. 3. It is supported by recent vLLM releases as Qwen3VLForConditionalGeneration. 4. It makes the assignment product-like: students can build an actual visual tutor, not only a synthetic text benchmark. 5. It leaves room for real inference decisions: batching, replicas, prefix caching, chunked prefill, eager vs compiled mode, request limits, and GPU budgeting.

Fixed Workloads

Everyone must benchmark on the same prompt set. This keeps the leaderboard fair.

The starter kit includes prompts.json with three workload types:

Workload	What it tests	Example
text	Normal tutor questions,	Explain why decode is me
long	Larger prefills, chunked	Design a debugging playb
image	Vision encoder path, mul	Inspect a toy dashboard
mixed	Product-like blend of al	The official multimodal

How the load tester builds each workload:

- long requests attach a synthetic vLLM server log (long_trace_lines in prompts.json, about 3k tokens) that is different for every request. Only the system prompt is shared, so prefix caching can't hide the prefill cost, and max_num_batched_tokens / chunked prefill actually matter.
- image requests send a 1024x768 diagram whose pixels differ slightly per request, so vLLM's multimodal caches can't skip the vision encoder. At 786k pixels it exceeds the default --mm-max-pixels, so that knob has a real effect.
- Metrics cover only the steady-state window after --ramp-up. ITL p95 is computed over every inter-token gap, so stalls mid-stream count.

Do not change the prompt set for your submitted runs. You may create additional private experiments, but final submissions must use the provided prompts.

Primary Goal

Maximize the leaderboard score while staying under the GPU budget and keeping errors low.

The official score is:

```
Score =
  goodput_tokens_per_second
  * sustained_users
  * quality_pass_rate
  * (1 - error_rate)
  / (p95_TTFT_seconds * p95_ITL_seconds * total_GPU_count)
```

For the starter harness, throughput is reported as streamed content chunks per second. In the final leaderboard, the evaluator may tokenize outputs with the Qwen tokenizer to compute exact output tokens per second. Your relative tuning strategy should be the same either way.

The score rewards:

- More useful output per second.
- More sustained concurrent users.
- Lower p95 time to first token.
- Lower p95 inter-token latency.
- Fewer errors.
- Better GPU efficiency.

The score penalizes:

- Throwing GPUs at the problem without improving efficiency.
- High p95 latency.
- Failed requests.
- Optimizations that improve text speed but break multimodal traffic.

Competition Tracks

Note on baselines: the 1-, 2- and 4-GPU numbers below were re-measured on 2026-11-03 with the current harness (H100, vLLM 0.21.0, 0 errors in ~15,600 requests), with the load generator on a laptop over the internet. ITL p95 here is measured per token at the client, so it includes network bunching: vLLM itself spaced tokens about 5 ms apart in compiled mode and 22-26 ms in eager mode. The 8-GPU table is from the earlier dry run with the old harness. Notice that the 4-replica mixed run does worse than the 2-replica one; finding out why is a good first experiment (look at server_metrics).

Track 1: Multimodal Product Track

This is the main capstone track.

Required workload:

```
--mode mixed
```

Recommended GPU budget:

```
Maximum 4 H100 GPUs
```

Reference baseline (re-measured 2026-11-03):

Config	Users	GPUs	Error rate	TTFT p95	ITL p95	Throughput
eager, 4 replicas, seq3	120	4	0.0%	2,655 ms	124.4 ms	2,618 chunks/s
eager, 2 replicas, seq3	100	2	0.0%	1,125 ms	91.4 ms	2,695 chunks/s

Your job is to beat this.

Track 2: Text Speed Track

This is the pure serving optimization track.

Required workload:

```
--mode text
```

Recommended GPU budget:

```
Maximum 4 H100 GPUs
```

Reference baseline (re-measured 2026-11-03):

Config	Users	GPUs	Error rate	TTFT p95	ITL p95	Throughput
compiled, 4 replicas, s	400	4	0.0%	1,167 ms	30.5 ms	18,476 chunks/s
compiled, 1 replica, se	100	1	0.0%	1,786 ms	38.5 ms	3,349 chunks/s

Compiled mode can be excellent for text-only serving, but it performed poorly on mixed multimodal traffic in our dry run. Use it deliberately.

Optional Boss Fight: 8 GPU Run

You may try up to 8 H100s if GPU quota is available.

Internal dry-run caveat:

Config	Users	GPUs	Error rate	TTFT p95	ITL p95	Throughput
compiled, 8 replicas	300	8	0.0%	1,116.1 ms	16.3 ms	11,502 chunks/s
compiled, 8 replicas	500	8	3.8%	1,581.2 ms	16.5 ms	15,251 chunks/s
compiled, 8 replicas	800	8	13.4%	2,257.0 ms	20.1 ms	15,927 chunks/s

The 8 GPU run produced high throughput, but errors appeared at higher user counts. This is an excellent advanced lesson: GPU math is not the only bottleneck. Web concurrency, request queues, cold starts, and server limits matter.

What You Are Allowed To Change

You may change:

- Number of replicas.
- GPU count and GPU type, within the assignment budget.
- max_num_seqs.
- max_num_batched_tokens.
- max_model_len.
- Prefix caching on/off.
- Chunked prefill on/off.
- Eager vs compiled mode.
- Concurrent request limit.
- Max output tokens, only if the track rules allow it.
- Image pixel budget, only if quality remains acceptable.
- Request ramp-up strategy during private experiments.
- Warmup strategy.

You may not change for final submission:

- The model family, unless a special track is announced.
- The official prompt set.
- The scoring script.
- The quality/evaluation rules.
- The workload mode for the track you submit to.

Optimization Hints

Start with systems reasoning, not random knob turning.

If TTFT is high

Try:

- Lowering max_num_batched_tokens.
- Enabling prefix caching.
- Enabling chunked prefill.
- Reducing image pixel budget for multimodal traffic.
- Adding replicas.
- Warming containers before the final run.

Ask yourself:

- Is prefill queueing behind long prompts?
- Are images too large?
- Did a replica cold start during the run?
- Are too many users hitting too few replicas?

If ITL is high

Try:

- Lowering max_num_seqs.
- Lowering max_num_batched_tokens.
- Using compiled mode for text-only runs.
- Reducing per-container concurrency.
- Separating long and short traffic, if you implement routing.

Ask yourself:

- Is each decode step too large?
- Are long prefills interrupting decode?
- Is the GPU saturated or underutilized?

If throughput is low

Try:

- Increasing users until the GPU is saturated.
- Increasing replicas.
- Increasing max_num_seqs, carefully.
- Using compiled mode for text-only track.
- Measuring whether errors are silently reducing goodput.

Ask yourself:

- Is the GPU idle because the load test is too small?
- Is the model waiting on image preprocessing?
- Is the endpoint rejecting requests?

If errors appear

Try:

- Reducing users.
- Reducing per-container concurrency.
- Increasing replicas.
- Increasing request timeout.
- Slowing ramp-up.
- Checking Modal logs.

Errors are not a side issue. The score penalizes them because real systems do not win by dropping user requests.

Required Submission

Submit a folder containing:

1. Your final benchmark JSON. 2. A one-page engineering report. 3. The exact command used for the final run. 4. A table of at least five experiments. 5. A short explanation of your best configuration.

Your report must answer:

- What was your final score?
- What was your best TTFT p95?
- What was your best ITL p95?
- What was your best throughput?
- What was your total GPU count?
- Which optimization helped the most?
- Which optimization failed or surprised you?
- What would you try next with more time?

Minimum Experiment Matrix

Run at least these experiments:

Experiment	Required comparison
Baseline	1 replica, default start
More users	Find where p95 latency b
Prefix caching	On vs off
Chunked prefill	On vs off
Batch knobs	seq32/b4096 vs one wider
Scale out	1 replica vs 2 or 4 repl
Final	Your best clean run

Attached Materials

The capstone package includes:

- Main assignment PDF: this document.
- Modal/vLLM setup PDF: what Modal is, how to authenticate, how vLLM is launched, and cost estimates.
- Starter code:
- modal_infertutor_app.py
- run_infertutor_experiment.py
- load_test_infertutor.py
- score_infertutor.py
- prompts.json
- requirements.txt
- README.md

The goal is that infrastructure should not be the hard part. Your time should go into inference engineering: reading metrics, forming hypotheses, changing serving parameters, and explaining why the results moved.